

Lecture 10: Neural Networks

Multi-Layer Perceptrons and the Backpropagation Algorithm

Caltech CS156: Learning from Data

September 18, 2026

1 Motivation: Beyond Linear Separability

To learn complex non-linear targets (such as XOR or arbitrary closed regions), single perceptrons are insufficient. By cascading perceptrons in layers, feedforward neural networks form complex decision boundaries through compositions of hyperplanes.

To enable gradient-based optimization across multiple layers, we replace the hard step function $\text{sgn}(s)$ with a smooth, differentiable soft threshold, typically the hyperbolic tangent:

$$\theta(s) = \tanh(s) = \frac{e^s - e^{-s}}{e^s + e^{-s}}, \quad \text{with derivative } \theta'(s) = 1 - \theta(s)^2 \quad (1)$$

2 Network Notation & Forward Pass

Consider an L -layer feedforward network:

- $l \in \{0, 1, \dots, L\}$: Layer index ($l = 0$ is input, $l = L$ is output).
- $d^{(l)}$: Number of neurons in layer l (excluding bias $x_0^{(l)} = 1$).
- $w_{ij}^{(l)}$: Weight connecting neuron i in layer $l - 1$ to neuron j in layer l .
- $s_j^{(l)}$: Linear signal at neuron j in layer l .
- $x_j^{(l)}$: Activation output of neuron j in layer l .

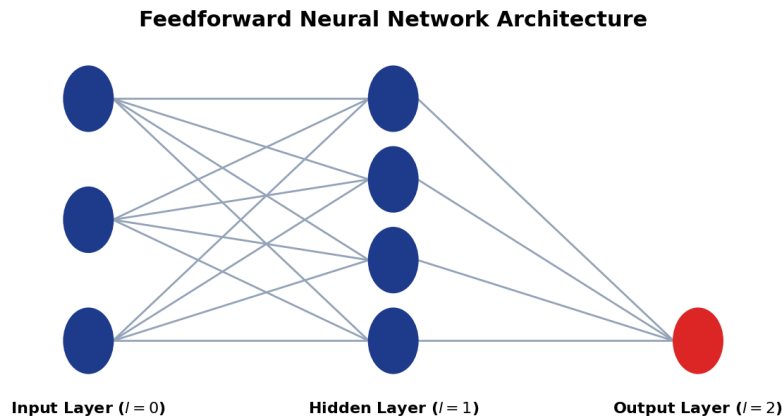


Figure 1: Architecture of a 3-layer Feedforward Neural Network.

2.1 Forward Propagation Equations

For each layer $l = 1, \dots, L$ and neuron $j = 1, \dots, d^{(l)}$:

$$s_j^{(l)} = \sum_{i=0}^{d^{(l-1)}} w_{ij}^{(l)} x_i^{(l-1)} \quad (2)$$

$$x_j^{(l)} = \theta(s_j^{(l)}) \quad (\text{with } x_0^{(l)} = 1) \quad (3)$$

3 The Backpropagation Algorithm

For a single instance $(\mathbf{x}_n, y_n)$ with squared error $e_n = (x_1^{(L)} - y_n)^2$, we compute partial derivatives $\frac{\partial e_n}{\partial w_{ij}^{(l)}}$ using the chain rule:

$$\frac{\partial e_n}{\partial w_{ij}^{(l)}} = \frac{\partial e_n}{\partial s_j^{(l)}} \cdot \frac{\partial s_j^{(l)}}{\partial w_{ij}^{(l)}} = \delta_j^{(l)} x_i^{(l-1)} \quad (4)$$

where $\delta_j^{(l)} \equiv \frac{\partial e_n}{\partial s_j^{(l)}}$ represents the **sensitivity/error signal** of neuron j in layer l .

Backpropagation Recurrence Equations

1. **Output Layer Sensitivity** ($l = L$):

$$\delta_1^{(L)} = 2(x_1^{(L)} - y_n) \cdot \theta'(s_1^{(L)}) \quad (5)$$

2. **Hidden Layer Sensitivity** ($l = L - 1, \dots, 1$):

$$\delta_i^{(l-1)} = \theta'(s_i^{(l-1)}) \sum_{j=1}^{d^{(l)}} w_{ij}^{(l)} \delta_j^{(l)} \quad (6)$$

4 Stochastic Gradient Descent (SGD) & Initialization

1. **Random Initialization:** Weights MUST be initialized randomly to small non-zero values. Initializing all weights to zero causes symmetric activations, rendering hidden neurons identical.
2. **Stochastic Update Step:** Pick a random instance $(\mathbf{x}_n, y_n)$, execute forward pass, perform backward pass to compute all $\delta_j^{(l)}$, and update weights:

$$w_{ij}^{(l)} \leftarrow w_{ij}^{(l)} - \eta x_i^{(l-1)} \delta_j^{(l)} \quad (7)$$